Advanced SQL-Like Query Language Compiler Frontend



KARNAVATI UNIVERSITY

UNITEDWORLD INSTITUTE OF TECHNOLOGY (UIT)

MINI PROJECT REPORT

ON

"Advanced SQL-Like Query Language Compiler Frontend"

Submitted as a part of the course

Compiler Design

Submitted By

Student Name	Enrollment No.
Rachit Barapatre	202307020155
Manthan Solanki	202307020153

Program: B.Tech CSE-AIML
Semester: VII

Under the Guidance of

Ms Neha Minocha

Assistant Professor

Department of Computer Science & Engineering Technology

Karnavati University, Gandhinagar

TABLE OF CONTENTS

1. Abstract 3
2. Introduction 3
3. Overview of Project 5
4. Technology Used 8
5. Flow Chart of the Project 8
6. Screen Shots of the Project 10
7. Results and Conclusion 13
References 15

List of Figures

Figure 5.1 Flow chart of the compiler frontend 8
Figure 6.1 Application home screen with token stream 9
Figure 6.2 Abstract Syntax Tree output 10
Figure 6.3 Query with a WHERE clause on string data 10
Figure 6.4 Lexical, syntax and semantic error screens 11

List of Tables

Table 3.1 Token categories recognised by the lexer 5
Table 4.1 Technologies and their role 7
Table 7.1 Test cases and observed output 11

1. ABSTRACT

A compiler translates a program written in a high-level language into a form that a machine, or another program, can act on. Its first half, called the frontend, reads raw source text and checks that it is well formed and meaningful. This mini project implements a complete compiler frontend for a small SQL-like query language. It follows the three classical analysis phases of compiler design: lexical analysis, syntax analysis and semantic analysis, and it produces an Abstract Syntax Tree (AST) as the intermediate result.

The system is written in Python. The lexical analyzer uses regular expressions to break a query into tokens such as keywords, identifiers, operators, numbers and strings. A hand-written recursive descent parser then checks the token stream against a formal grammar for the `SELECT ... FROM ... [WHERE ...] [ORDER BY ...];` statement and builds the AST as a nested dictionary. A semantic analyzer finally rejects queries that are grammatically correct but logically wrong, such as repeated column names in the SELECT list.

A Streamlit web interface lets a user type a query, run all three phases with a single button and inspect the token table, the AST in JSON form, the number of tokens and the compilation time. Errors are reported with a message that names the phase in which they were detected. Fourteen test queries were run against the implementation: valid queries compile in under one millisecond, and each invalid query is stopped by the correct phase with an understandable message. The project demonstrates how the theory of regular expressions, context-free grammars and syntax-directed analysis turns into working software.

Keywords: compiler design, lexical analysis, recursive descent parsing, abstract syntax tree, semantic analysis, SQL, Python, Streamlit.

2. INTRODUCTION

2.1 Background

Every programming language processor, whether a compiler, an interpreter or a database engine, must first understand the text it is given. A database server that receives the string `SELECT name FROM students;` cannot execute it until it has recognised the words, confirmed that they are arranged in a legal order and decided what they mean. These are exactly the jobs of the frontend of a compiler. The backend, which optimises and generates code or executes the plan, can only work on input that the frontend has already validated.

The frontend of a compiler is conventionally divided into three analysis phases. **Lexical analysis** groups characters into the smallest meaningful units, called tokens. **Syntax analysis** checks that the sequence of tokens follows the grammar of the language and builds a tree that captures the structure of the statement. **Semantic analysis** checks rules that a grammar alone cannot express, such as whether names are used consistently. Each phase receives the output of the previous one and either passes a cleaner representation forward or stops with an error.

2.2 Motivation

SQL is a language that every computer science student meets early, which makes it a good vehicle for studying compilers: the vocabulary is small and familiar, the structure is clear, and the effect of an error is easy to see. Building a frontend for a subset of SQL allows each compiler phase to be implemented in full, without the volume of detail that a general-purpose language such as C would bring. It also shows that the same techniques sit behind real tools: query engines, code editors and linters all begin by tokenising and parsing their input.

2.3 Problem Statement

To design and implement a compiler frontend for a SQL-like query language that accepts a text query, performs lexical, syntax and semantic analysis, generates an Abstract Syntax Tree for valid queries, and reports a clear, phase-specific error message for invalid ones, all through an interactive interface.

2.4 Objectives

- To implement a lexical analyzer that converts a query into a classified stream of tokens.
- To define a formal grammar for the supported query language and implement a recursive descent parser for it.
- To generate an Abstract Syntax Tree that represents the structure of every valid query.
- To add a semantic analysis phase that detects logical errors that survive parsing.

2.5 Scope and Limitations

The project covers the analysis half of a compiler only. It does not generate code, optimise a query plan or connect to a real database, so a query is checked and represented but never executed. The supported language is a single `SELECT` statement with a column list, one table, an optional single-condition `WHERE` clause and an optional `ORDER BY` clause. Section 7.3 lists the limitations in more detail.

3. OVERVIEW OF PROJECT

3.1 System Architecture

The application is a single Python file with four logical parts: the lexical analyzer, the parser, the semantic analyzer and the user interface. The parts form a pipeline. The text of the query enters the lexer, which returns a list of tokens. The parser consumes the tokens and returns an AST. The semantic analyzer inspects the AST and either approves it or returns an error message. The interface runs the pipeline when the user presses the compile button and displays whatever the pipeline produced. If any phase fails, later phases are not run, and any previously stored tokens and AST are cleared so that stale results are never shown next to a new error.

3.2 Supported Language and Grammar

The language accepted by the compiler is defined by the following grammar. Terminals are written in capital letters, optional parts are in square brackets and repetition is shown with braces.

```
<query>        ::= SELECT <column_list> FROM IDENTIFIER
                   [ <where_clause> ] [ <order_clause> ] SEMICOLON
<column_list>  ::= IDENTIFIER { COMMA IDENTIFIER }
<where_clause> ::= WHERE IDENTIFIER OPERATOR ( NUMBER | STRING )
<order_clause> ::= ORDER BY IDENTIFIER [ ASC | DESC ]
```

Keywords are case-insensitive, so `select`, `Select` and `SELECT` are equivalent. The comparison operators are =, >, <, >=, <= and `!=`. An `ORDER BY` clause defaults to ascending order when no direction is given.

3.3 Phase 1: Lexical Analyzer

The lexical analyzer is built from an ordered list of token specifications, each a name paired with a regular expression. The expressions are joined into one master pattern with named groups, and `re.finditer` scans the query from left to right. The order of the list matters because the first alternative that matches wins: keywords are tried before identifiers so that `FROM` is not mistaken for a column name, and two-character operators such as >= are tried before single-character ones. Table 3.1 summarises the categories.

Table 3.1: Token categories recognised by the lexer

Token type	Pattern (summary)	Example
KEYWORD	SELECT, FROM, WHERE, ORDER BY, BY, ASC, DESC, AND, OR	`SELECT`, `ORDER BY`

Token type	Pattern (summary)	Example
OPERATOR	Two-character >=, <=, != and single >, <, =	>=, =
NUMBER	One or more digits	21
STRING	Text inside single or double quotes	'Ahmedabad'
IDENTIFIER	Letter or underscore followed by letters, digits or underscores	name, students
COMMA / SEMICOLON	The characters , and ;	, ;
SKIP	Spaces, tabs and newlines (discarded)	(none)
MISMATCH	Any other character; raises a lexical error	@, *

Whitespace is discarded. Multi-word keywords such as `ORDER BY` are normalised to a single upper-case token whatever spacing the user typed. If a character matches no rule, the lexer stops at once and returns a lexical error that names the offending character. For the default query the lexer produces sixteen tokens.

3.4 Phase 2: Syntax Analyzer and AST Generation

The parser is a recursive descent parser with one token of lookahead. It keeps a position in the token list and provides two helpers: `peek()` looks at the current token without moving, and `consume()` checks that the current token has the expected type or value, moves forward and returns it, or raises a `SyntaxError` that says what was expected and what was found. The `parse()` method follows the grammar rule by rule. It consumes `SELECT`, reads a comma-separated list of identifiers, consumes `FROM` and the table name, and then uses `peek()` to decide whether a `WHERE` clause and an `ORDER BY` clause follow. Finally it requires a semicolon and rejects any token left after it.

While it parses, the parser fills a dictionary that serves as the AST. For the default query the tree is:

```
{
  "action":   "SELECT",
  "fields":   ["name", "age", "city"],
  "table":    "students",
  "condition": {"field": "age", "operator": ">=", "value": "21"},
  "order_by":  {"field": "age", "direction": "DESC"}
}
```

The `condition` and `order_by` entries exist only if the matching clause was present, so later phases can test for them directly. This tree is the intermediate representation that a backend would use to plan and execute the query.

3.5 Phase 3: Semantic Analyzer

A query can follow the grammar and still be meaningless. The semantic analyzer inspects the finished AST for two such cases. The first check rejects duplicate column names in the SELECT list, since selecting the same column twice is ambiguous. The second check is a simulated schema rule: the column named in the `WHERE` condition must not have the same name as the target table. The function returns `None` when the tree is acceptable and an error string otherwise. The same structure can later be extended with a real schema lookup.

3.6 Error Handling Strategy

The compiler is fail-fast: the first error stops processing. Each phase reports errors in its own vocabulary, which helps the user locate the mistake. A lexical error names an illegal character. A syntax error states which token was expected and which was found, or that the query ended unexpectedly. A semantic error describes the broken rule in plain words. The error prefix, "Lexical", "Syntax" or "Semantic", identifies the phase.

3.7 User Interface

The interface is split into two columns. The left column holds a text area for the query, pre-filled with an example, and a button that runs the whole frontend. The right column shows the result: a success banner with three metrics (status, number of tokens and compilation time in milliseconds), or a red error banner. Below the columns, two tabs present the token stream as a table and the AST as an expandable JSON view.

4. TECHNOLOGY USED

The project deliberately uses a small set of tools so that the compiler logic remains easy to read. Table 4.1 lists them and states the role of each.

Table 4.1: Technologies and their role

Technology	Type	Role in the project
Python 3	Language	Implements the lexer, parser and semantic analyzer. Dictionaries and lists make the token stream and AST easy to build and inspect.
`re` module	Standard library	Compiles the token specification into one master regular expression and scans the query with `finditer`.
Streamlit	Web framework	Builds the interactive interface (text area, button, metrics, tabs, JSON viewer) without separate HTML or JavaScript.
`time` module	Standard library	Measures compilation time in milliseconds.
`json` / session state	Library / feature	Presents the AST in JSON form and stores results between Streamlit reruns.

Why regular expressions for the lexer. Token patterns are regular languages, and a regular expression is the direct, compact way to describe them. Python's named groups also report which rule matched, which removes the need for a hand-written state machine.

Why recursive descent for the parser. The grammar is small and has no left recursion, so each rule maps onto a few lines of code, and the error messages come naturally from `consume()`. A table-driven parser generator would be heavier than the problem needs.

Why Streamlit. The goal of the project is the compiler, not the front end of a website. Streamlit turns a Python script into a working web application and re-runs it on every interaction, which suits a tool whose output is a function of its input.

5. FLOW CHART OF THE PROJECT

Figure 5.1 shows how a query moves through the compiler. Three decision points correspond to the three analysis phases, and every error exit leads straight to the end of the run.

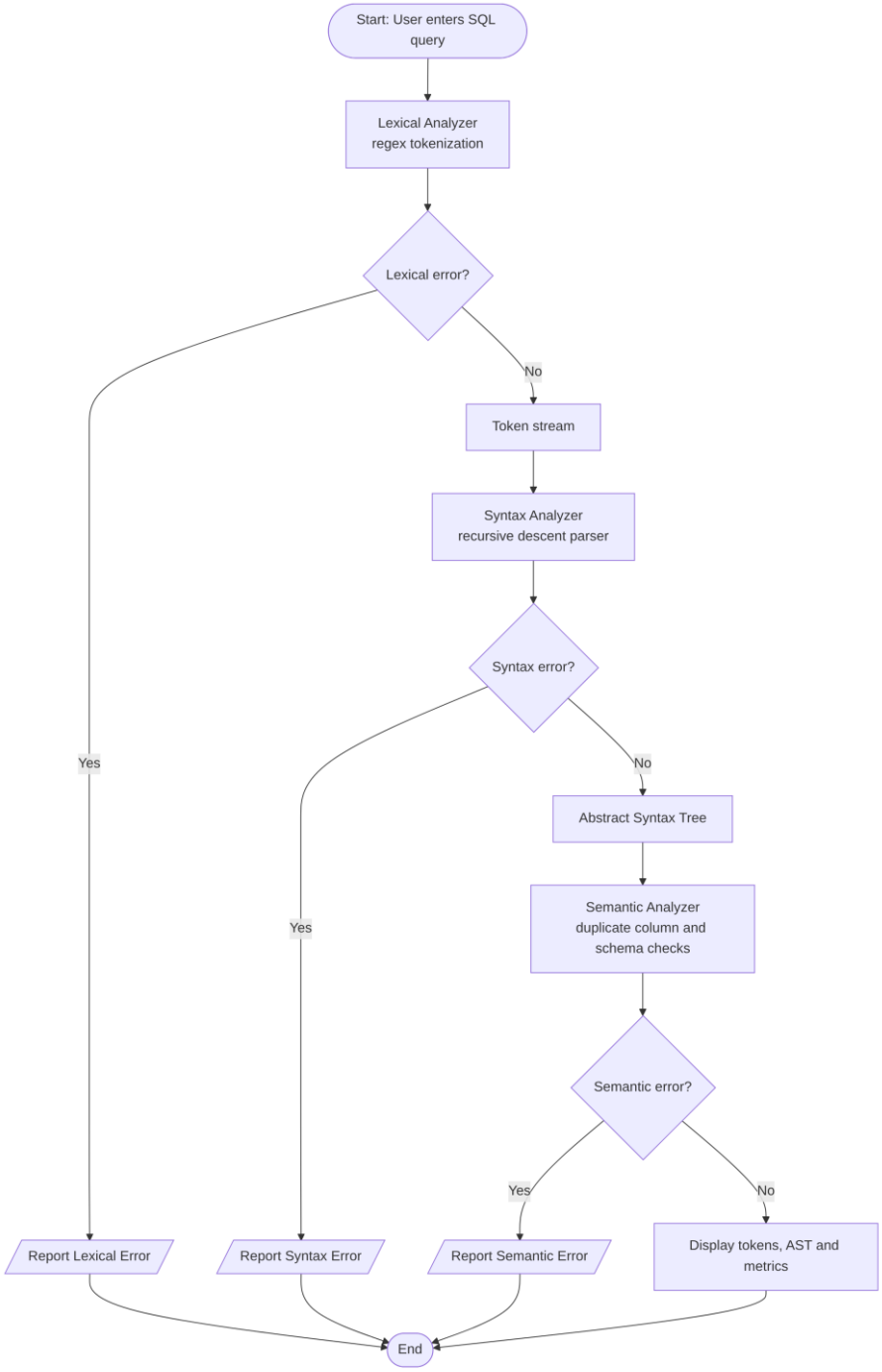


Figure 5.1: Flow chart of the compiler frontend

6. SCREEN SHOTS OF THE PROJECT

6.1 Successful Compilation

Figure 6.1 shows the application after the default query was compiled. The right panel reports a successful status, sixteen tokens and a compilation time of well under a millisecond. The Phase 1 tab lists each token with its type and value.

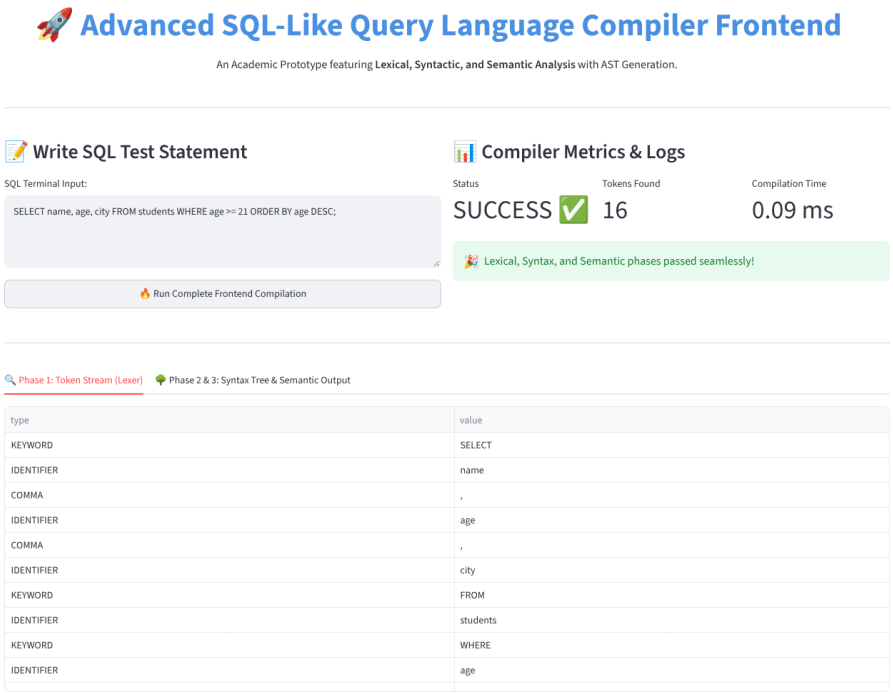


Figure 6.1: Application home screen with token stream

Figure 6.2 shows the second tab, which presents the Abstract Syntax Tree built by the parser. The tree matches the structure given in Section 3.4.

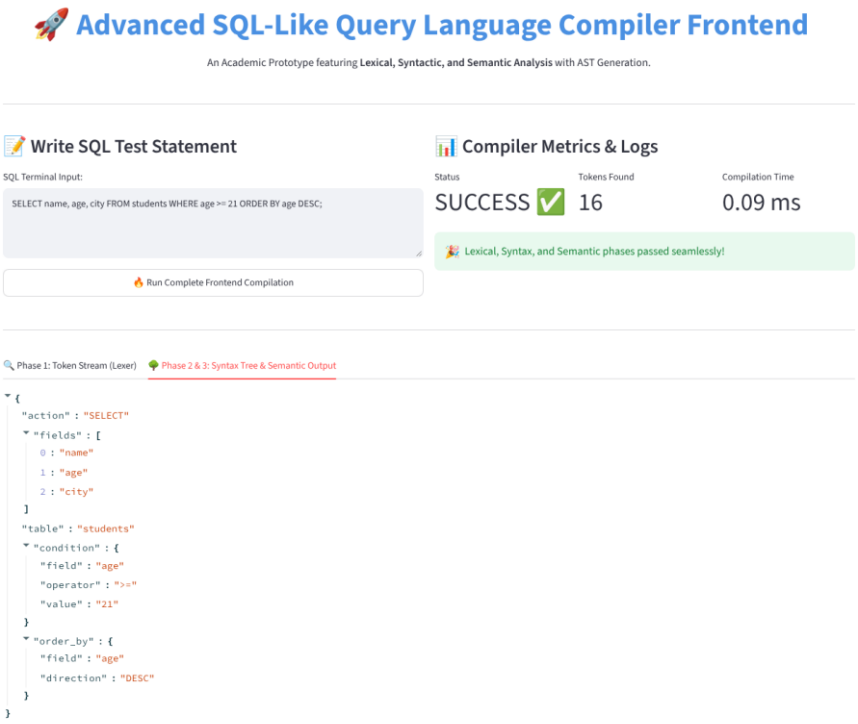


Figure 6.2: Abstract Syntax Tree output

Figure 6.3 shows a different valid query that filters on a string value and has no `ORDER BY` clause. The AST for this query contains a `condition` entry but no `order_by` entry.

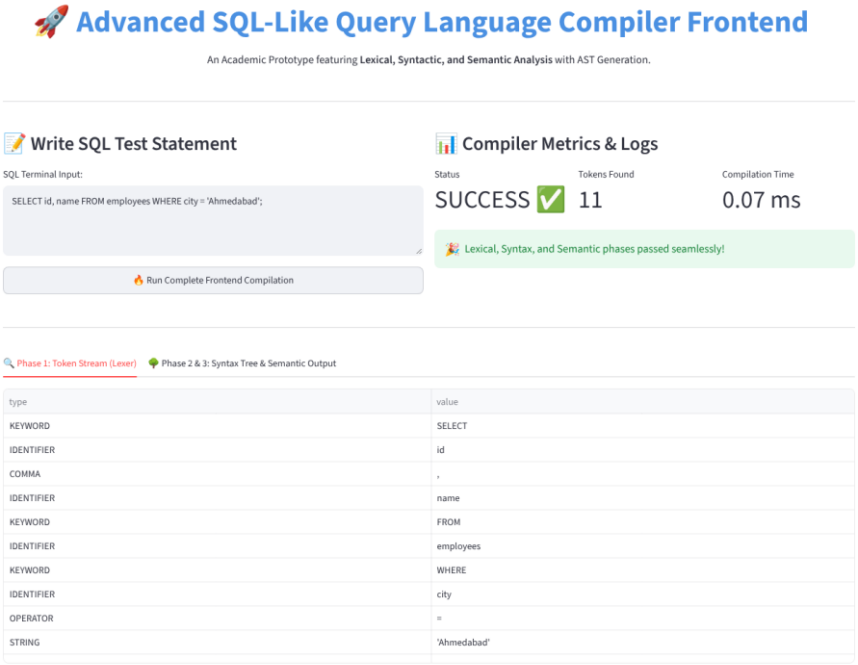


Figure 6.3: Query with a WHERE clause on string data

6.2 Error Reporting

Figure 6.4 combines three screens, one for each analysis phase: an illegal character caught by the lexer, a missing table name caught by the parser and a duplicate column caught by the semantic analyzer.

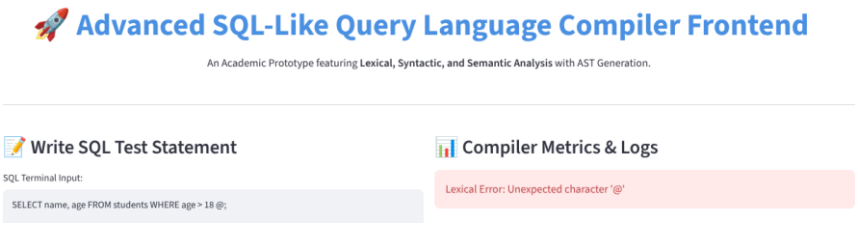


Figure 6.4(a): Lexical error: unexpected character

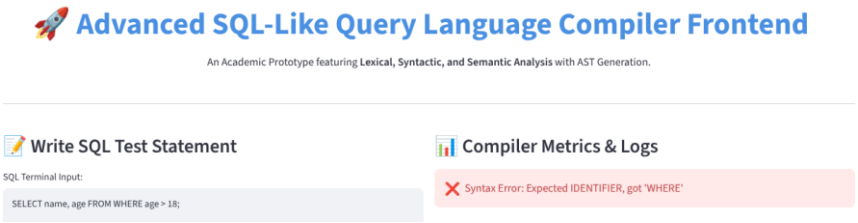


Figure 6.4(b): Syntax error: expected IDENTIFIER, got WHERE

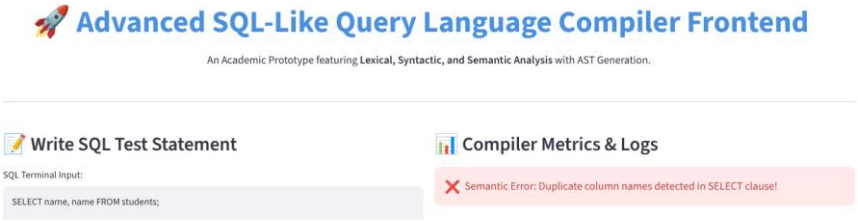


Figure 6.4(c): Semantic error: duplicate column names

7. RESULTS AND CONCLUSION

7.1 Test Cases and Results

The three analysis functions were run on fourteen queries covering valid statements and each class of error. Table 7.1 shows a representative selection together with the observed result. In every case the phase that stopped the query was the one expected.

Table 7.1: Test cases and observed output

Query	Outcome	Observed result
`SELECT name, age, city FROM students WHERE age >= 21 ORDER BY age DESC;`	Pass	Accepted; 16 tokens; AST generated
`SELECT id, name FROM employees WHERE city = 'Ahmedabad';`	Pass	Accepted; 11 tokens; AST generated
`select name from students;`	Pass	Accepted; lower-case keywords normalised; 5 tokens
`SELECT name, age FROM WHERE age > 18;`	Syntax	Expected IDENTIFIER, got 'WHERE'
`SELECT name FROM students`	Syntax	Unexpected end of query
`SELECT name FROM students WHERE age > ;`	Syntax	Expected Number or String in WHERE condition
`SELECT name FROM students; extra`	Syntax	Unexpected tokens after semicolon: 'extra'
`SELECT name, age FROM students WHERE age > 18 @;`	Lexical	Unexpected character '@'
`SELECT name, name FROM students;`	Semantic	Duplicate column names detected in SELECT clause
`SELECT name FROM students WHERE students = 5;`	Semantic	Column 'students' cannot have the same name as the target table

Compilation time for valid queries was measured at between 0.09 and 0.63 milliseconds on the test machine, with the first run slightly slower than later ones. At that speed the interface feels instantaneous, which is expected for queries of this length and for a one-pass scanner and parser.

7.2 Observations

- The ordering of rules in the lexer is critical. Placing keywords before identifiers and two-character operators before one-character operators prevents wrong tokenisation.

- Separating the phases made errors easy to localise. The error prefix alone tells the user whether to fix a character, the structure of the statement or the meaning of a name.
- Storing the AST as a plain dictionary keeps the representation transparent. Optional clauses simply appear or do not appear as keys.
- Clearing stored results after a failure avoids showing a token table that belongs to an earlier query.

7.3 Limitations

- `SELECT *` is not supported: the asterisk is rejected by the lexer as an unexpected character.
- The lexer recognises `AND` and `OR`, but the grammar allows only one `WHERE` condition, so a query with `AND` fails in the parser (expected semicolon, got 'AND').
- Only integer numbers are accepted; a decimal such as `3.5` produces a lexical error at the decimal point.
- Errors name the offending token but not its line and column position, and processing stops at the first error.
- The schema check is a simulation. Table and column names are not verified against a real catalogue, and the operands of a comparison are not type-checked.

7.4 Future Scope

- Extend the grammar with `SELECT *`, multiple conditions joined by `AND` and `OR`, decimal numbers, `GROUP BY` and `JOIN`.
- Add a real schema (tables, columns and types) so that the semantic analyzer can check names and compare operand types.
- Record line and column numbers in tokens and use panic-mode recovery so that several errors can be reported in one run.
- Add the backend: generate an intermediate representation or an execution plan from the AST and run it on sample data.

7.5 Conclusion

The project achieved its objectives. It delivers a working compiler frontend for a SQL-like language, with a regular-expression lexer, a recursive descent parser driven by an explicit grammar, an AST generator and a semantic analyzer, behind an interactive interface that makes

each phase visible. Valid queries are turned into a clear tree structure, and invalid ones are stopped with a message that identifies the phase and the cause.

Beyond the working tool, the project gave practical experience of the central ideas of compiler design: that the structure of a language can be defined formally, that a complex task is made manageable by splitting it into phases, and that good error reporting is part of a compiler's design, not an afterthought. The same pipeline, extended with more grammar rules and a backend, is the basis of real query processors.

REFERENCES

[1] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed., Pearson Education, 2006.

[2] K. D. Cooper and L. Torczon, *Engineering a Compiler*, 2nd ed., Morgan Kaufmann, 2011.

[3] Python Software Foundation, "re — Regular expression operations," Python 3 documentation, https://docs.python.org/3/library/re.html.

[4] Streamlit Inc., Streamlit documentation, https://docs.streamlit.io.